lenet_mnist_loss_surface

January 11, 2018

In [1]:
```
%pylab inline
import pickle
import copy
```

```
Populating the interactive namespace from numpy and matplotlib
```

In [2]:
```
from __future__ import print_function
import argparse
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.autograd import Variable

# Training settings
parser = argparse.ArgumentParser(description='PyTorch MNIST Example')
parser.add_argument('--batch-size', type=int, default=64, metavar='N',
                    help='input batch size for training (default: 64)')
parser.add_argument('--test-batch-size', type=int, default=1000, metavar='N
                    help='input batch size for testing (default: 1000)')
parser.add_argument('--epochs', type=int, default=10, metavar='N',
                    help='number of epochs to train (default: 10)')
parser.add_argument('--lr', type=float, default=0.01, metavar='LR',
                    help='learning rate (default: 0.01)')
parser.add_argument('--momentum', type=float, default=0.5, metavar='M',
                    help='SGD momentum (default: 0.5)')
parser.add_argument('--no-cuda', action='store_true', default=False,
                    help='disables CUDA training')
parser.add_argument('--seed', type=int, default=1, metavar='S',
                    help='random seed (default: 1)')
parser.add_argument('--log-interval', type=int, default=10, metavar='N',
                    help='how many batches to wait before logging training
args = parser.parse_args([])
args.cuda = not args.no_cuda and torch.cuda.is_available()

torch.manual_seed(args.seed)
```

```
        if args.cuda:
            print('Using CUDA')
            torch.cuda.manual_seed(args.seed)


        kwargs = {'num_workers': 1, 'pin_memory': True} if args.cuda else {}
        train_loader = torch.utils.data.DataLoader(
            datasets.MNIST('../data', train=True, download=True,
                           transform=transforms.Compose([
                               transforms.ToTensor(),
                               transforms.Normalize((0.1307,), (0.3081,))
                           ])),
            batch_size=args.batch_size, shuffle=True, **kwargs)
        test_loader = torch.utils.data.DataLoader(
            datasets.MNIST('../data', train=False, transform=transforms.Compose([
                               transforms.ToTensor(),
                               transforms.Normalize((0.1307,), (0.3081,))
                           ])),
            batch_size=args.test_batch_size, shuffle=True, **kwargs)
        class Net(nn.Module):
            def __init__(self):
                super(Net, self).__init__()
                self.conv1 = nn.Conv2d(1, 10, kernel_size=5)
                self.conv2 = nn.Conv2d(10, 20, kernel_size=5)
                self.conv2_drop = nn.Dropout2d()
                self.fc1 = nn.Linear(320, 50)
                self.fc2 = nn.Linear(50, 10)

            def forward(self, x):
                x = F.relu(F.max_pool2d(self.conv1(x), 2))
                x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))
                x = x.view(-1, 320)
                x = F.relu(self.fc1(x))
                x = F.dropout(x, training=self.training)
                x = self.fc2(x)
                return F.log_softmax(x, dim=1)
```

```
Using CUDA
Files already downloaded
```

```
In [3]: model = Net()
        if args.cuda:
            model.cuda()
        with open('lenet_models/lenet_1_00.p', 'wb') as fd:
            pickle.dump(model, fd)
        torch.save(model.state_dict(), 'lenet_models/lenet_00.pt')
        optimizer = optim.SGD(model.parameters(), lr=args.lr, momentum=args.momentu
```

```
        torch.save(optimizer.state_dict(),'lenet_models/optimizer.pt')
        def train(epoch):
            model.train()
            for batch_idx, (data, target) in enumerate(train_loader):
                if args.cuda:
                    data, target = data.cuda(), target.cuda()
                data, target = Variable(data), Variable(target)
                optimizer.zero_grad()
                output = model(data)
                loss = F.nll_loss(output, target)
                loss.backward()
                optimizer.step()
                # if batch_idx % args.log_interval == 0:
                    # print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.forma
                        # epoch, batch_idx * len(data), len(train_loader.dataset),
                        # 100. * batch_idx / len(train_loader), loss.data[0]))

        def test():
            model.eval()
            test_loss = 0
            correct = 0
            for data, target in test_loader:
                if args.cuda:
                    data, target = data.cuda(), target.cuda()
                data, target = Variable(data, volatile=True), Variable(target)
                output = model(data)
                test_loss += F.nll_loss(output, target, size_average=False).data[0]
                pred = output.data.max(1, keepdim=True)[1] # get the index of the m
                correct += pred.eq(target.data.view_as(pred)).cpu().sum()

            test_loss /= len(test_loader.dataset)
            print('Test set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%)'.forma
                test_loss, correct, len(test_loader.dataset),
                100. * correct / len(test_loader.dataset)))


        for epoch in range(1, args.epochs + 1):
            train(epoch)
            test()
            with open('lenet_models/lenet_1_%02d.p' % epoch, 'wb') as fd:
                pickle.dump(model, fd)
```

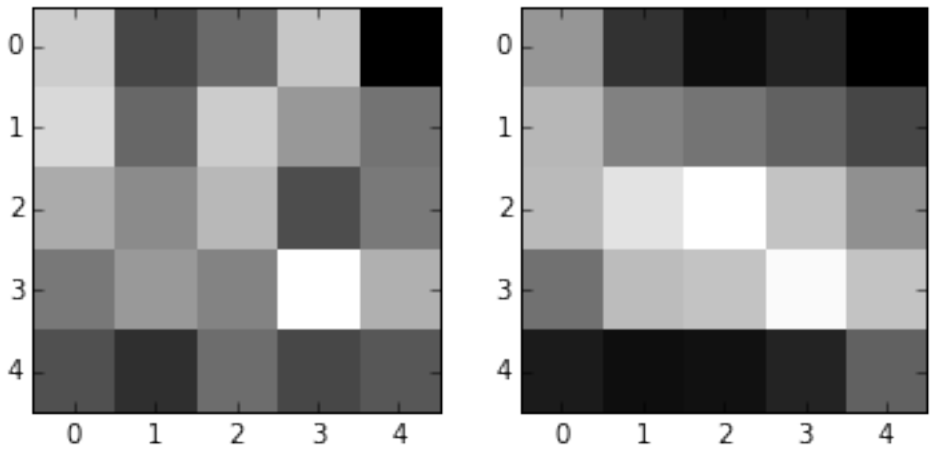
```
Test set: Average loss: 0.1962, Accuracy: 9430/10000 (94%)
Test set: Average loss: 0.1266, Accuracy: 9614/10000 (96%)
Test set: Average loss: 0.0955, Accuracy: 9690/10000 (97%)
Test set: Average loss: 0.0868, Accuracy: 9727/10000 (97%)
Test set: Average loss: 0.0717, Accuracy: 9774/10000 (98%)
Test set: Average loss: 0.0653, Accuracy: 9796/10000 (98%)
```

```
Test set: Average loss: 0.0633, Accuracy: 9798/10000 (98%)
Test set: Average loss: 0.0589, Accuracy: 9814/10000 (98%)
Test set: Average loss: 0.0546, Accuracy: 9834/10000 (98%)
Test set: Average loss: 0.0567, Accuracy: 9831/10000 (98%)
```

```
In [4]: with open('lenet_models/lenet_1_00.p', 'rb') as fd:
            model_init = pickle.load(fd)
        with open('lenet_models/lenet_1_10.p', 'rb') as fd:
            model_final = pickle.load(fd)
        d_init = next(model_init.parameters())[0,0]
        d_final = next(model_final.parameters())[0,0]
        print(d_init, d_final)
        subplot(1, 2, 1)
        imshow(d_init.data, interpolation='nearest', cmap='gray')
        subplot(1, 2, 2)
        imshow(d_final.data, interpolation='nearest', cmap='gray')
```

```
Variable containing:
 0.1031 -0.0883 -0.0388  0.0939 -0.1883
 0.1199 -0.0411  0.1017  0.0278 -0.0245
 0.0555  0.0099  0.0730 -0.0779 -0.0146
-0.0180  0.0290 -0.0008  0.1748  0.0622
-0.0745 -0.1208 -0.0335 -0.0863 -0.0641
[torch.cuda.FloatTensor of size 5x5 (GPU 0)]
 Variable containing:
 0.0934 -0.2028 -0.3104 -0.2485 -0.3541
 0.1906  0.0302 -0.0090 -0.0641 -0.1452
 0.2004  0.3252  0.4071  0.2313  0.0744
-0.0127  0.2060  0.2297  0.3921  0.2286
-0.2711 -0.3106 -0.3030 -0.2477 -0.0654
[torch.cuda.FloatTensor of size 5x5 (GPU 0)]
```

```
Out[4]: <matplotlib.image.AxesImage at 0x7f7e2179a278>
```




```
In [5]: for p in model_init.parameters():
            print(p.size())
```

```
torch.Size([10, 1, 5, 5])
torch.Size([10])
torch.Size([20, 10, 5, 5])
torch.Size([20])
torch.Size([50, 320])
torch.Size([50])
torch.Size([10, 50])
torch.Size([10])
```

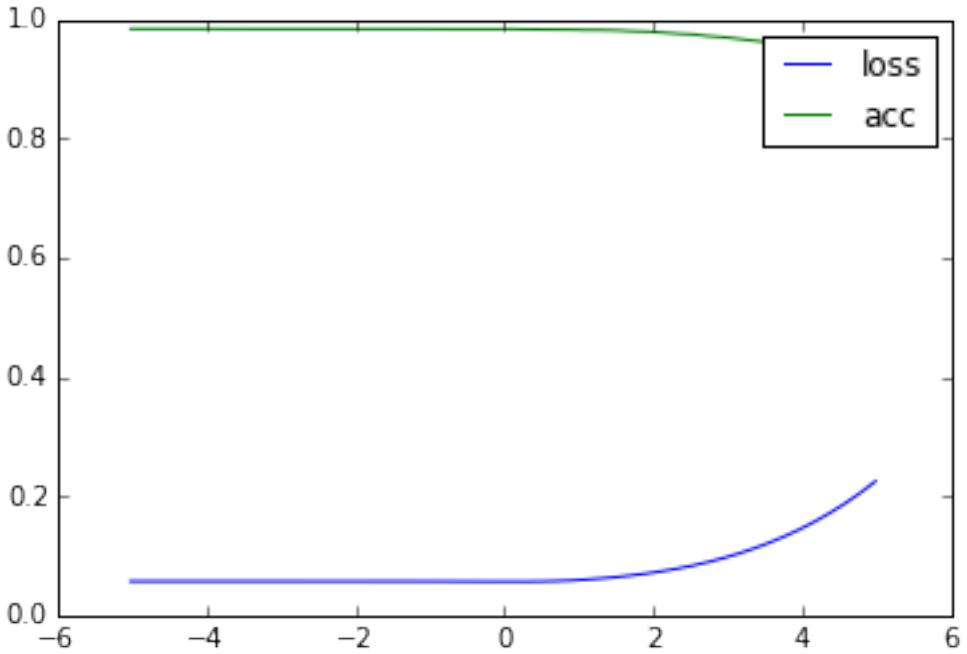
0.1 exp 1: Loss Surface

```
In [6]: std = 5
        prec = 50
        def test_model(model):
            model.eval()
            test_loss = 0
            correct = 0
            for data, target in test_loader:
                if args.cuda:
                    data, target = data.cuda(), target.cuda()
                data, target = Variable(data, volatile=True), Variable(target)
                output = model(data)
                test_loss += F.nll_loss(output, target, size_average=False).data[0]
                pred = output.data.max(1, keepdim=True)[1] # get the index of the m
                correct += pred.eq(target.data.view_as(pred)).cpu().sum()

            test_loss /= len(test_loader.dataset)
```

```
        return test_loss, correct / len(test_loader.dataset)
    with open('lenet_models/lenet_1_10.p', 'rb') as fd:
        model_final = pickle.load(fd)
    for ix, p in enumerate(model_final.parameters()):
        if ix == 2:
            param = p[4,2,2,4]
    param_v = param.data[0]
    param
```

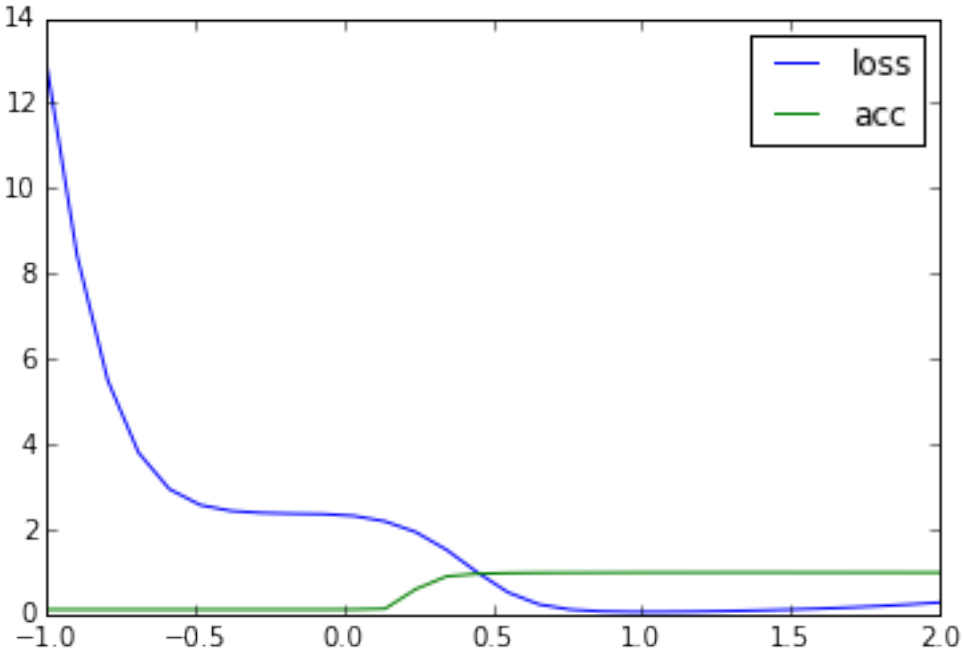
```
Out[6]: Variable containing:
        1.00000e-02 *
         -1.7142
        [torch.cuda.FloatTensor of size 1 (GPU 0)]
```

```
In [7]: v, loss, acc = [], [], []
        for i in linspace(param_v - std, param_v + std, prec):
            param.data[0] = i
            l, a = test_model(model_final)
            v.append(i)
            loss.append(l)
            acc.append(a)
        plot(v, loss, label='loss')
        plot(v, acc, label='acc')
        legend()
        show()
```



0.2 exp2 train difference

```
In [8]: params_base, params_diff = [], []
        with open('lenet_models/lenet_1_00.p', 'rb') as fd:
            model_init = pickle.load(fd)
        with open('lenet_models/lenet_1_10.p', 'rb') as fd:
            model_final = pickle.load(fd)
        for x, y in zip(model_init.parameters(), model_final.parameters()):
            params_diff.append(y - x)
            params_base.append(x.clone())
        params_diff[0][0,0]

Out[8]: Variable containing:
        -0.0096 -0.1145 -0.2716 -0.3424 -0.1658
         0.0706  0.0713 -0.1108 -0.0919 -0.1207
         0.1449  0.3153  0.3341  0.3092  0.0890
         0.0053  0.1770  0.2305  0.2173  0.1663
        -0.1967 -0.1898 -0.2695 -0.1615 -0.0013
        [torch.cuda.FloatTensor of size 5x5 (GPU 0)]

In [9]: v, loss, acc = [], [], []
        for i in linspace(-1, 2, 30):
            i = float(i)
            for p, b, d in zip(model_init.parameters(), params_base, params_diff):
                p.data[...] = (b + i * d).data
            l, a = test_model(model_init)
            v.append(i)
            loss.append(l)
            acc.append(a)
        plot(v, loss, label='loss')
        plot(v, acc, label='acc')
        legend()
        show()
```



0.3 exp 3: train with same init and different data order

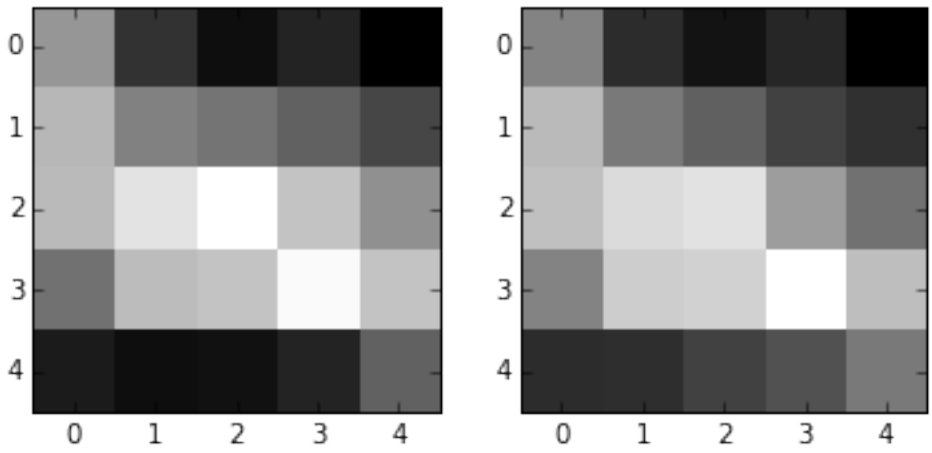
```
In [10]: model.load_state_dict(torch.load('lenet_models/lenet_00.pt'))
         optimizer.load_state_dict(torch.load('lenet_models/optimizer.pt'))
         with open('lenet_models/lenet_2_00.p', 'wb') as fd:
             # should be same with lenet_1_00.p
             pickle.dump(model, fd)
         for epoch in range(1, args.epochs + 1):
             train(epoch)
             test()
             with open('lenet_models/lenet_2_%02d.p' % epoch, 'wb') as fd:
                 pickle.dump(model, fd)
```

```
Test set: Average loss: 0.2008, Accuracy: 9395/10000 (94%)
Test set: Average loss: 0.1238, Accuracy: 9627/10000 (96%)
Test set: Average loss: 0.1017, Accuracy: 9674/10000 (97%)
Test set: Average loss: 0.0828, Accuracy: 9745/10000 (97%)
Test set: Average loss: 0.0777, Accuracy: 9761/10000 (98%)
Test set: Average loss: 0.0668, Accuracy: 9784/10000 (98%)
Test set: Average loss: 0.0601, Accuracy: 9808/10000 (98%)
Test set: Average loss: 0.0583, Accuracy: 9815/10000 (98%)
Test set: Average loss: 0.0581, Accuracy: 9824/10000 (98%)
Test set: Average loss: 0.0531, Accuracy: 9833/10000 (98%)
```

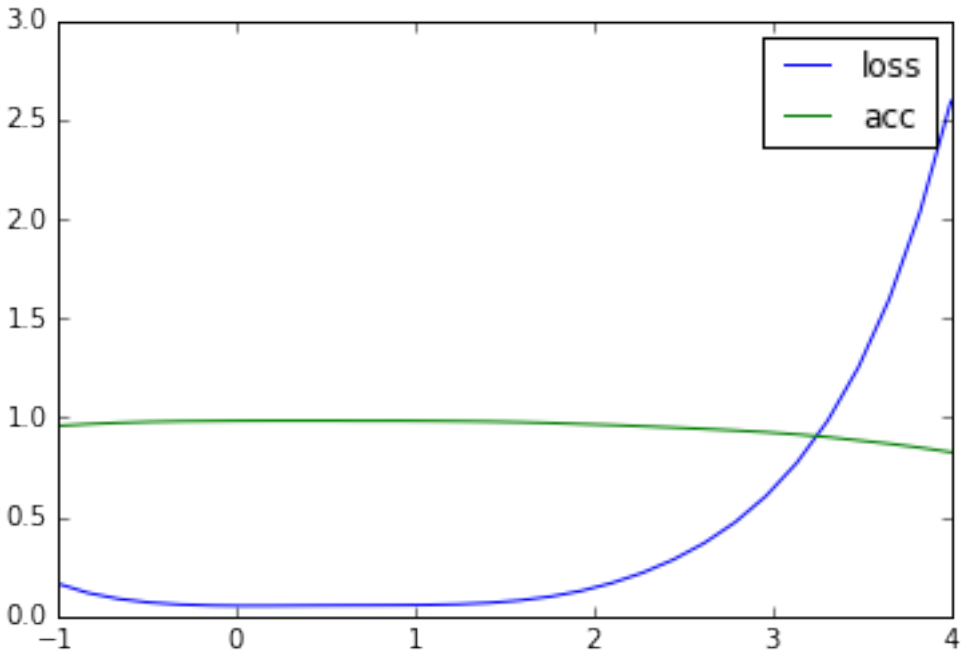
In [11]:
```
# val & plot
with open('lenet_models/lenet_1_00.p', 'rb') as fd:
    model_init = pickle.load(fd)
with open('lenet_models/lenet_1_10.p', 'rb') as fd:
    model_1 = pickle.load(fd)
with open('lenet_models/lenet_2_10.p', 'rb') as fd:
    model_2 = pickle.load(fd)

params_base, params_diff1, params_diff2 = [], [],[]
for x, y1, y2 in zip(model_init.parameters(), model_1.parameters(), model_
    params_diff1.append(y1 - x)
    params_diff2.append(y2 - x)
    params_base.append(x.clone())
print(params_diff1[0][0,0], params_diff2[0][0,0])
subplot(1, 2, 1)
imshow(next(model_1.parameters())[0,0].data, interpolation='nearest', cmap
subplot(1, 2, 2)
imshow(next(model_2.parameters())[0,0].data, interpolation='nearest', cmap
```

```
Variable containing:
-0.0096 -0.1145 -0.2716 -0.3424 -0.1658
 0.0706  0.0713 -0.1108 -0.0919 -0.1207
 0.1449  0.3153  0.3341  0.3092  0.0890
 0.0053  0.1770  0.2305  0.2173  0.1663
-0.1967 -0.1898 -0.2695 -0.1615 -0.0013
[torch.cuda.FloatTensor of size 5x5 (GPU 0)]
 Variable containing:
-0.0672 -0.1378 -0.2647 -0.3415 -0.1748
 0.0783  0.0466 -0.1745 -0.1928 -0.1917
 0.1608  0.2881  0.2445  0.1877 -0.0044
 0.0499  0.2267  0.2684  0.2327  0.1468
-0.1560 -0.1027 -0.1330 -0.0310  0.0702
[torch.cuda.FloatTensor of size 5x5 (GPU 0)]
```

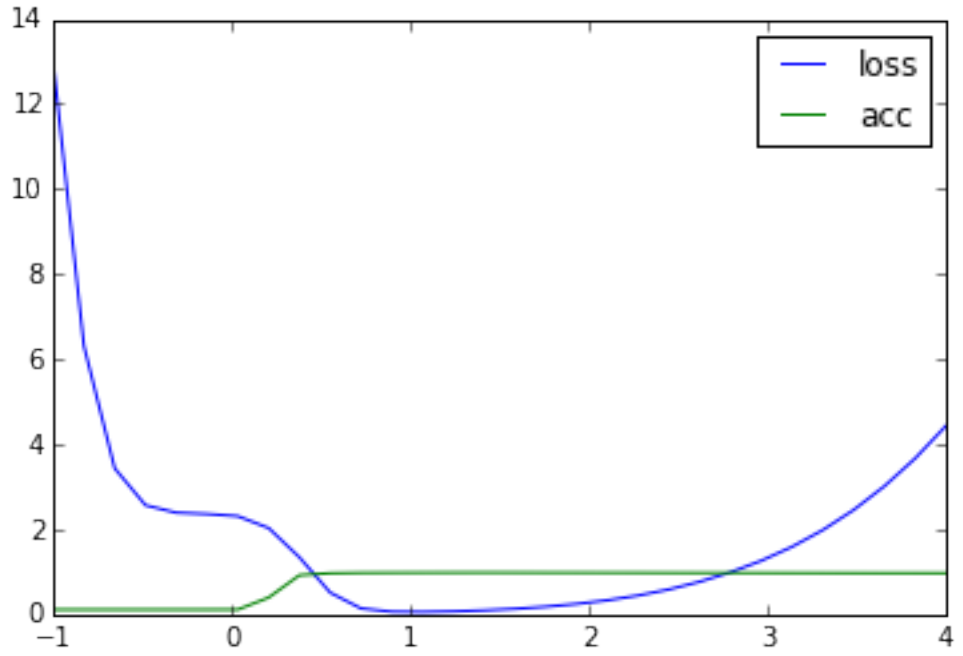
Out[11]:
```
<matplotlib.image.AxesImage at 0x7f7e215ba160>
```




```
In [12]: def between(x, y, begin=0, end=1, prec=30):
             model = copy.deepcopy(x)
             v, loss, acc = [], [], []
             for i in linspace(begin, end, prec):
                 i = float(i)
                 for p, b, d in zip(model.parameters(), x.parameters(), y.parameter
                     p.data[...] = (b*(1-i) + d * i).data
                 l, a = test_model(model)
                 v.append(i)
                 loss.append(l)
                 acc.append(a)
             return v, loss, acc

In [13]: v, l, a = between(model_2, model_1, -1, 4)
         plot(v, l, label='loss')
         plot(v, a, label='acc')
         legend()
         show()
```



In [14]:
```
v, l, a = between(model_init, model_1, -1, 4)
plot(v, l, label='loss')
plot(v, a, label='acc')
legend()
show()
```

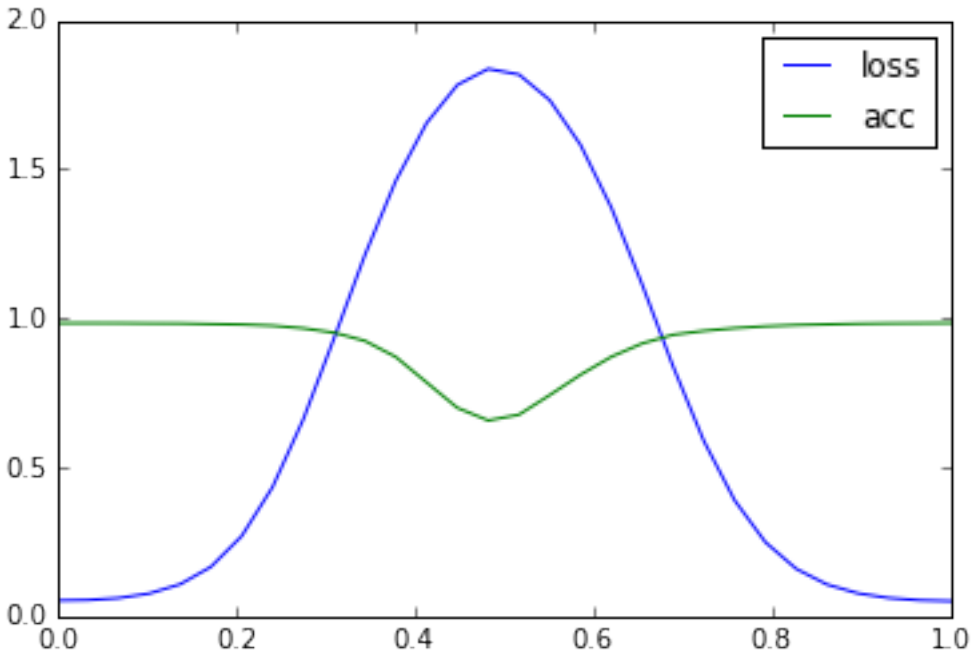


0.4 exp 4: train from different parameter

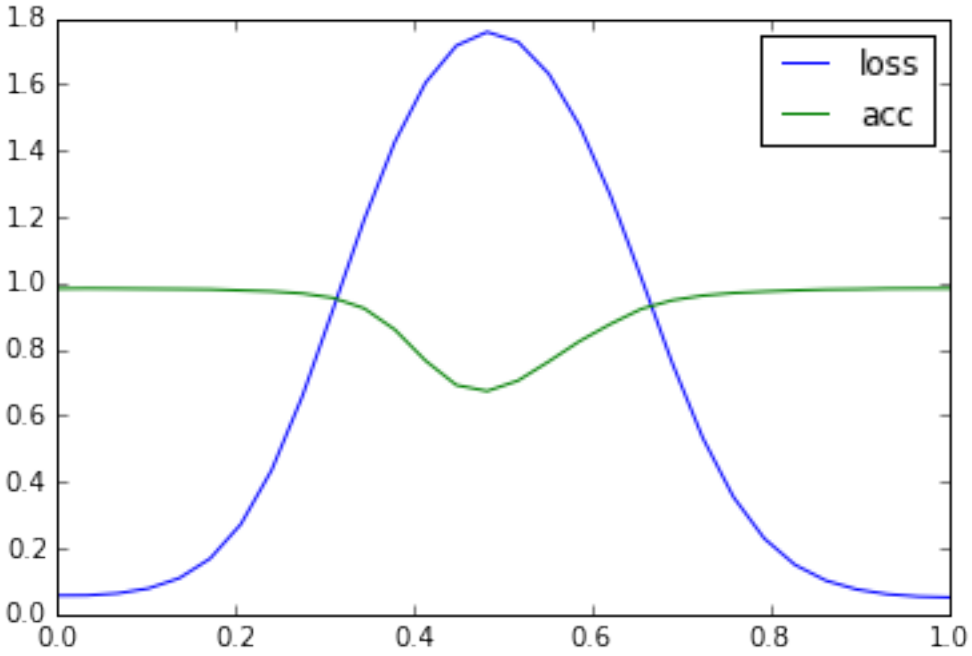
```
In [15]: for i in model.parameters():
             if len(i.size()) > 1:
                 nn.init.xavier_normal(i)
             else:
                 i.data.fill_(0)

         optimizer.load_state_dict(torch.load('lenet_models/optimizer.pt'))
         with open('lenet_models/lenet_3_00.p', 'wb') as fd:
             pickle.dump(model, fd)
         for epoch in range(1, args.epochs + 1):
             train(epoch)
             test()
             with open('lenet_models/lenet_3_%02d.p' % epoch, 'wb') as fd:
                 pickle.dump(model, fd)
```

```
Test set: Average loss: 0.2023, Accuracy: 9435/10000 (94%)
Test set: Average loss: 0.1212, Accuracy: 9639/10000 (96%)
Test set: Average loss: 0.0960, Accuracy: 9706/10000 (97%)
Test set: Average loss: 0.0813, Accuracy: 9749/10000 (97%)
Test set: Average loss: 0.0720, Accuracy: 9779/10000 (98%)
Test set: Average loss: 0.0672, Accuracy: 9778/10000 (98%)
Test set: Average loss: 0.0569, Accuracy: 9824/10000 (98%)
Test set: Average loss: 0.0567, Accuracy: 9824/10000 (98%)
Test set: Average loss: 0.0511, Accuracy: 9840/10000 (98%)
Test set: Average loss: 0.0514, Accuracy: 9837/10000 (98%)
```

```
In [16]: with open('lenet_models/lenet_2_10.p', 'rb') as fd:
             model_2 = pickle.load(fd)
         with open('lenet_models/lenet_3_10.p', 'rb') as fd:
             model_3 = pickle.load(fd)
         v, l, a = between(model_2, model_3)
         plot(v, l, label='loss')
         plot(v, a, label='acc')
         legend()
         show()
```




```
In [17]: with open('lenet_models/lenet_1_10.p', 'rb') as fd:
             model_1 = pickle.load(fd)
         with open('lenet_models/lenet_3_10.p', 'rb') as fd:
             model_3 = pickle.load(fd)
         v, l, a = between(model_1, model_3)
         plot(v, l, label='loss')
         plot(v, a, label='acc')
         legend()
         show()
```



0.5 exp 5: the space without normalize

```
In [ ]: def grid(z, x, y, begin=0, end=1, prec=30):
            model = copy.deepcopy(z)
            loss, acc = np.empty([prec, prec]), np.empty([prec, prec])
            v = linspace(begin, end, prec)
            for ix, i in enumerate(v):
                i = float(i)
                vt, losst, acct = [], [], []
                for iy, j in enumerate(v):
                    j = float(j)
                    for p, b, d, q in zip(model.parameters(), z.parameters(), x.par
                        p.data[...] = (b * (1 - i - j) + d * i + q * j).data
                    l, a = test_model(model)
                    loss[ix, iy] = l
                    acc[ix, iy] = a
            return v, loss, acc
```

```
In [ ]: v, l, a = grid(model_init, model_2, model_3, 0, 3, 200)
        figure(figsize=(12, 8))
        ax=subplot(1, 2, 1)
        imshow(l, interpolation='nearest')
        ax.set_xticklabels(v)
        ax.set_yticklabels(v)
        colorbar()
```

```
ax=subplot(1, 2, 2)
imshow(a, interpolation='nearest')
ax.set_xticklabels(v)
ax.set_yticklabels(v)
colorbar()
savefig('out.png')
show()
```